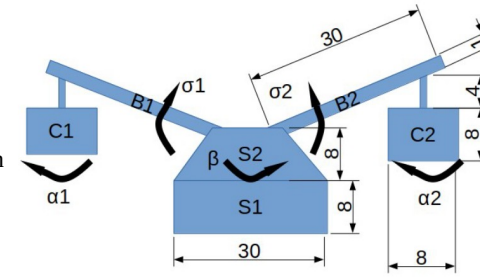
Examen Informática Gráfica (19/01/24) Grupo A

Apellidos:_____ Nombre _____

1. Diseñar el grafo de escena del modelo que se muestra en la figura.

Las canastas C1 y C2 giran en torno a su eje Y, los brazos B1 y B2 giran independientemente. El soporte S2 gira con respecto al eje vertical, moviendo los brazos. Las canastas (C1 y C2) son cubos; los brazos, el soporte S1 y las uniones de las canastas con los brazos son cilindros; y la pieza S2 un tronco de cono. Se parte de un cubo de lado 1 centrado en el origen; un cilindro, de radio 1 y altura 1, centrado en el origen; y un tronco de cono centrado, de radio inferior 1, radio superior 0.5 y altura 1.



Evaluación: construcción de las piezas (hasta 1 pto.), modelo estático (hasta 1 pto.), modelo dinámico (hasta 1 pto.) (total hasta 3 ptos.)

2. Se desea incluir una valla publicitaria digital de 5m de ancho por 2.5 de alto en una escena. La valla ha de mostrar dos imágenes alternas en el tiempo, intercambiándolas cada 10 segundos . El cambio de una imagen a otra se realiza mediante un desplazamiento vertical que dura un segundo. El objeto que representa la valla es una malla representada por la estructura del ejercicio anterior, con 2 triángulos y 4 vértices. Asumiendo que se dispone de una imagen con los dos anuncios como la mostrada, explica como se puede conseguir el efecto de animación de la malla y escribe el pseudocodigo de la función de animación asumiendo que dicha función se llama cada décima segundo de segundo.

Evaluación: planteamiento del problema (hasta 1 pto.), codificación (hasta 1pto.) (total hasta 2 ptos.)

3. Se tiene una escena compuesta por M objetos representados por mallas cada una con N triángulos. Explica como se pueden asignar y codificar los identificadores para que usando el método de selección por color se obtenga el número de objeto y el número de triángulo en la malla.

Evaluación: obtención del número del objeto (hasta 1 pto.) obtención del número del triángulo del objeto (hasta 1 put.) (hasta 2 ptos.)

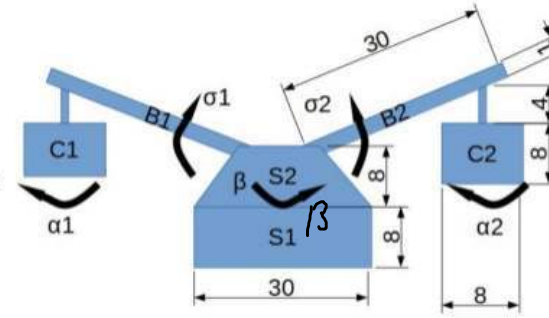
4. Describa el cauce convencional de visualización de OpenGL indicando lo que se hace en cada etapa **(hasta 1 pto.)**

5. Dado un objeto definido por sus vértices y triángulos, explica como se podría comprobar que el objeto es cerrado. Implementa la idea mediante una función en C++.

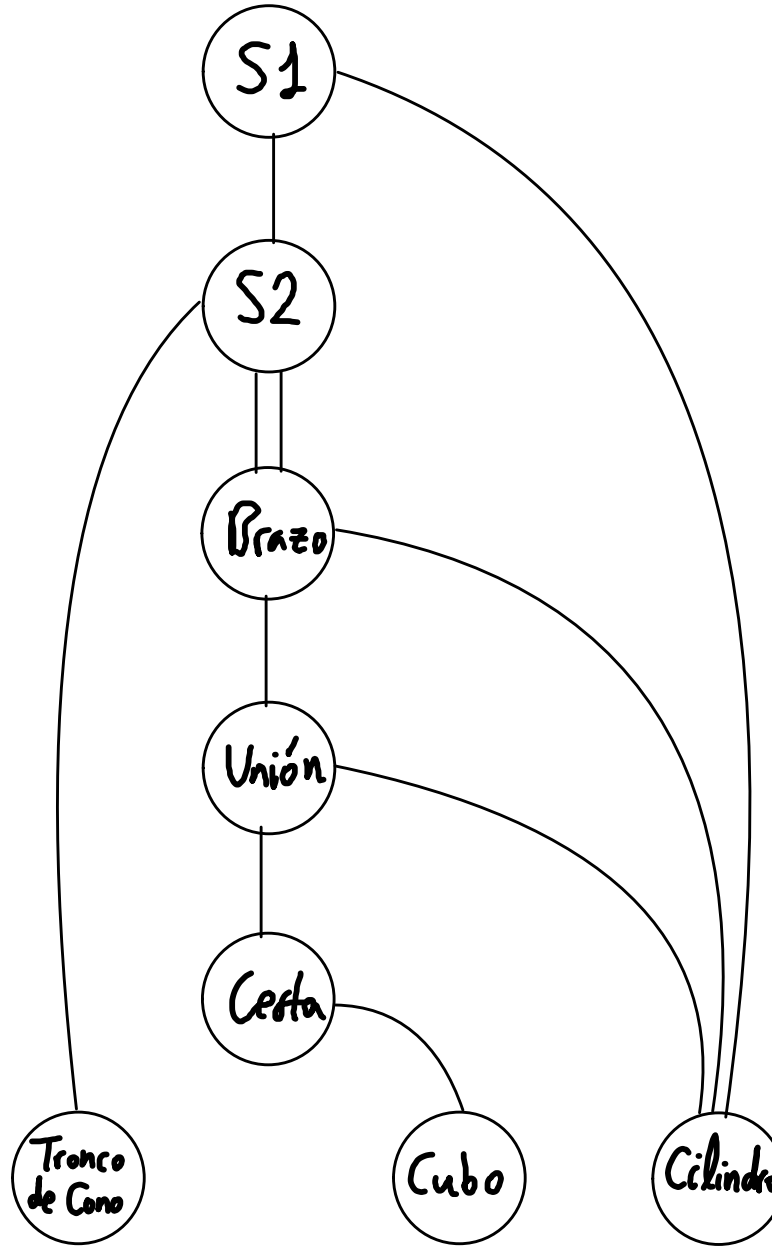
Evaluación: planteamiento del problema (hasta 1 pto.), codificación (hasta 1pto.) (total hasta 2 ptos.)

1. Diseñar el grafo de escena del modelo que se muestra en la figura.

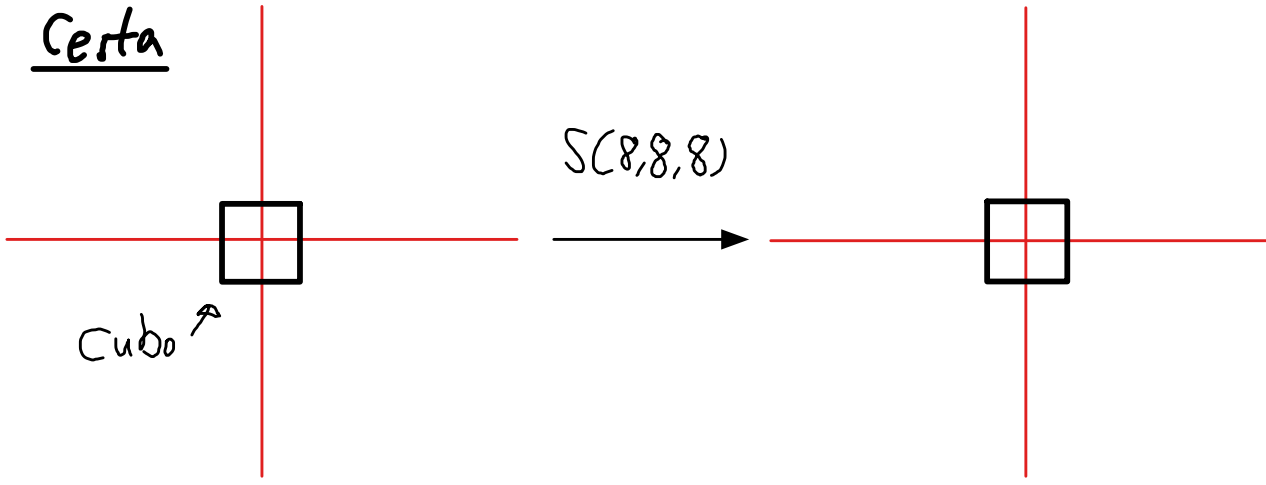
Las canastas C1 y C2 giran en torno a su eje Y, los brazos B1 y B2 giran independientemente. El soporte S2 gira con respecto al eje vertical, moviendo los brazos. Las canastas (C1 y C2) son cubos; los brazos, el soporte S1 y las uniones de las canastas con los brazos son cilindros; y la pieza S2 un tronco de cono. Se parte de un cubo de lado 1 centrado en el origen; un cilindro, de radio 1 y altura 1, centrado en el origen; y un tronco de cono centrado, de radio inferior 1, radio superior 0.5 y altura 1.



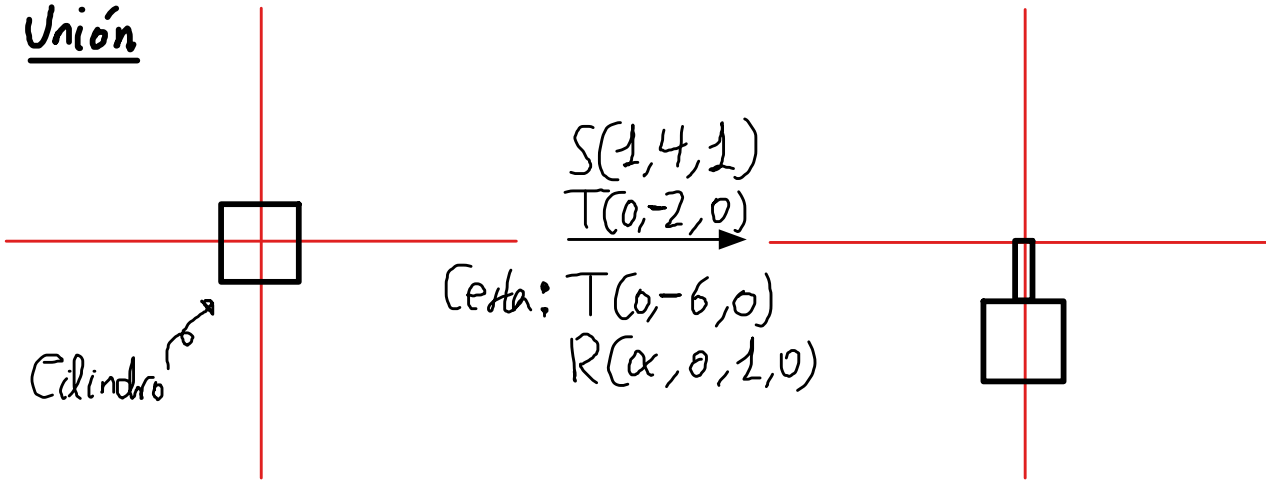
Grafo



Cesta

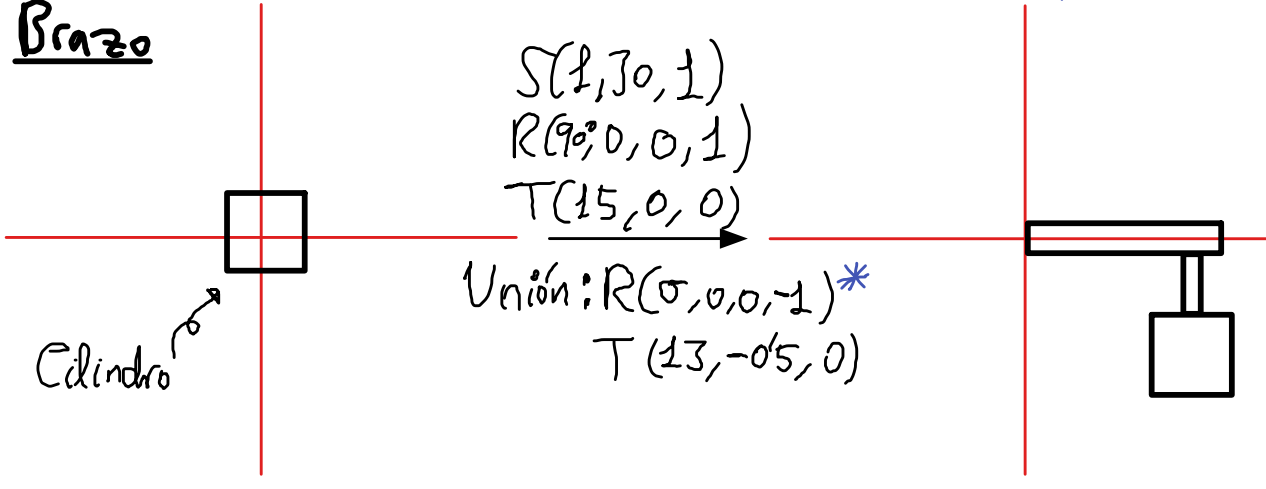


Unión

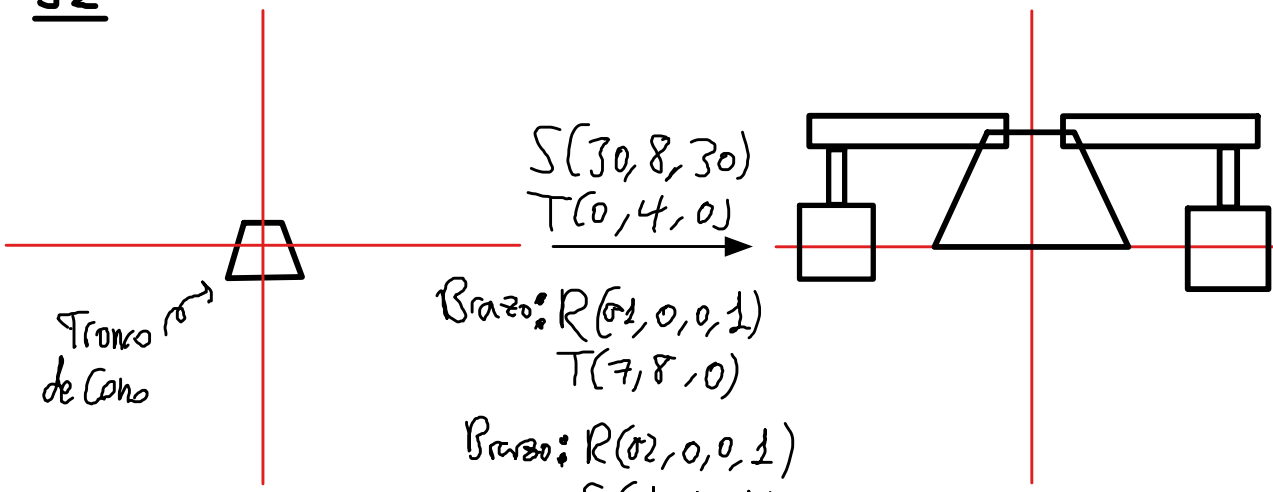


*Para que la cesta siempre apunte al suelo.

Brazo



S2

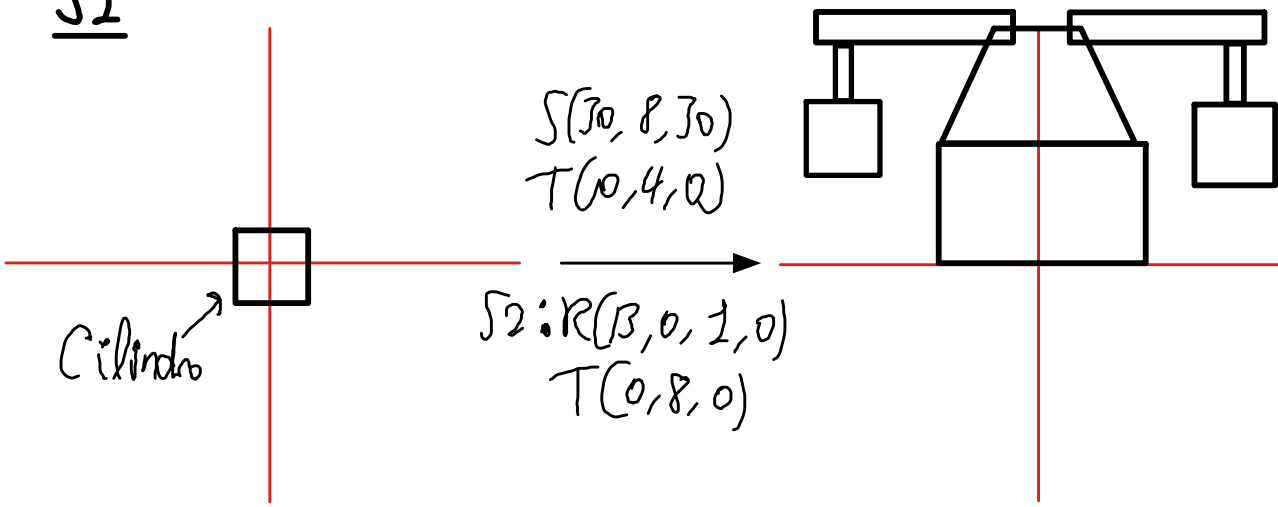


S(30, 8, 30)
T(0, 4, 0)

Brazo: R(-1, 0, 0, 1)
T(7, 8, 0)

Brazo: R(-2, 0, 0, 1)
S(-1, 1, 1)
T(-7, 8, 0)

S1



S(30, 8, 30)
T(0, 4, 0)

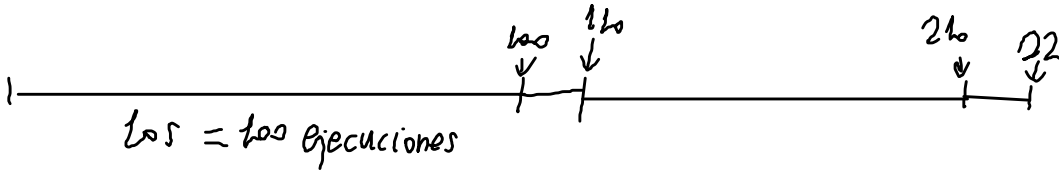
S2: R(3, 0, 1, 0)
T(0, 8, 0)

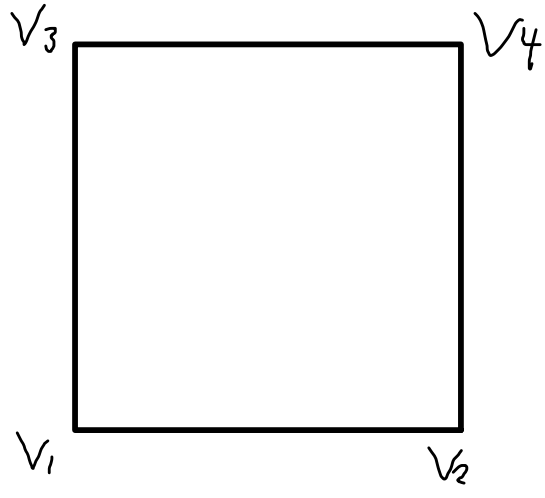
2. Se desea incluir una valla publicitaria digital de 5m de ancho por 2.5 de alto en una escena. La valla ha de mostrar dos imágenes alternas en el tiempo, intercambiándolas cada 10 segundos . El cambio de una imagen a otra se realiza mediante un desplazamiento vertical que dura un segundo. El objeto que representa la valla es una malla representada por la estructura del ejercicio anterior, con 2 triángulos y 4 vértices. Asumiendo que se dispone de una imagen con los dos anuncios como la mostrada, explica como se puede conseguir el efecto de animación de la malla y escribe el pseudocodigo de la función de animación asumiendo que dicha función se llama cada décima segundo de segundo.

Evaluación: planteamiento del problema (hasta 1 pto.), codificación (hasta 1pto.) (total hasta 2 ptos.)

Para lograr el efecto de la animación hay que cambiar las coordenadas de textura gradualmente siguiendo la siguiente fórmula, R(t)= (1-t)*P0 + t*P1 siendo t = i/10, con i como las veces que se ejecutó la función.

Para ello podemos definir variables estáticas para registrar el progreso y también temporizar el cambio.





```c/c++
void animacion(vector<_vertex3f>& tex){
	static int contador = 0;
	_vertex2f P0[4], P1[4];

	if(contador => 100 && contador < 110){
		P0 = (_vertex2f(0,0.5),_vertex2f(1,0.5),
			_vertex2f(0,1),_vertex2f(1,1));
		P1 = (_vertex2f(0,0),_vertex2f(1,0),
			_vertex2f(0,0.5),_vertex2f(1,0.5));
		float t = (contador - 100)/10.0f;


	}else if(contador => 210 && contador < 220){
		P1 = (_vertex2f(0,0.5),_vertex2f(1,0.5),
			_vertex2f(0,1),_vertex2f(1,1));
		P0 = (_vertex2f(0,0),_vertex2f(1,0),
			_vertexf2f(0,0.5),_vertex2f(1,0.5));
		float t = (contador - 210)/10.0f;
	}else if(contador >= 220){
		contador = 0;
	}

	tex.resize(4);
	for(int i = 0; i < 4; i++){
		tex[i].x = P0[i].x * (1-t) + P1[i].x*t;
		tex[i].y = P0[i].y * (1-t) + P1[i].y*t;
	}
}
```

3. Se tiene una escena compuesta por M objetos representados por mallas cada una con N triángulos. Explica como se pueden asignar y codificar los identificadores para que usando el método de selección por color se obtenga el número de objeto y el número de triángulo en la malla.

Evaluación: obtención del número del objeto (hasta 1 pto.) obtención del número del triángulo del objeto (hasta 1 put.) (hasta 2 ptos.)

Primero asignamos IDs de 0 a M−1 a los objetos.

Hacemos lo mismo con los triángulos de cada objeto de 0 a N−1.

Ahora cada triángulo recibe un valor RGB según su ID y el objeto al que pertenece.

Color = (ID_objeto * N + ID_triángulo)

Después:

R = Color & 0X00FF0000

G = Color & 0X0000FF00

B = Color & 0X000000FF

Así, a la hora del dibujado cada triángulo tendrá un color diferente.

Cuando el usuario selecciona un pixel en la pantalla leemos el color del pixel y calculamos el ID.

$$ID = R \cdot 2^{16} + G \cdot 2^{8} + B$$

Para terminar obtenemos el ID del objeto y el ID del triángulo.

$$ID_objeto = ID / N$$

$$ID_triángulo = ID \% N$$

4. Describa el cauce convencional de visualización de OpenGL indicando lo que se hace en cada etapa **(hasta 1 pto.)**

1.Transformación de vértices: Se procesan las vértices aplicando transformaciones con matrices de modelo, vista y proyección para posicionar las vértices en el espacio de pantalla.

2.Ensamblado y rasterización de primitivas: Los vértices transformados se agrupan en primitivas(triángulos, líneas o puntos) que luego se rasterizan, generando fragmentos(píxeles candidatos).

3.Procesamiento de fragmentos: Se calcula el color final de cada fragmentos con las texturas y la iluminación.

4.Escritura en el framebuffer: Los fragmentos pasan por pruebas como profundidad(z-buffer) para determinar los válidos, estos son finalmente escritos en el framebuffer creando la imagen final que se muestra en pantalla.

5. Dado un objeto definido por sus vértices y triángulos, explica como se podría comprobar que el objeto es cerrado. Implementa la idea mediante una función en C++.

Evaluación: planteamiento del problema (hasta 1 pto.), codificación (hasta 1pto.) (total hasta 2 ptos.)

Para comprobar que el objeto está cerrado tenemos que comprobar que cada línea pertenece a dos triángulos.
Si el objeto está bien construido, una línea como mucho estará en dos triángulos, así que construiremos un vector donde iremos almacenando las líneas de cada triángulo , y cuando encontremos una línea que ya existía en el vector la eliminamos. Al terminar de recorrer todos los triángulos, si queda alguna línea en el vector, significa que el objeto está abierto

```C/C++
//Vector de vértices
vector<_vertex3f> Vertices;
//Vector de triángulos
vector<_vertex3ui> Triangulos;

bool esCerrado(){
	//Usamos unordered_set en vez del vector por eficiencia
	unordered_set<_vertex2ui> Lineas_unicas;
	_vertex2ui lineas[3];

	for(size_t i = 0; i < Triangulos.size(); i++){
		lineas[0] = _vertex2ui(Triangulos[i].x, Triangulos[i].y;
		lineas[1] = _vertex2ui(Triangulos[i].y, Triangulos[i].z;
		lineas[2] = _vertex2ui(Triangulos[i].z, Triangulos[i].x;

		for(size_t j = 0; j < 3; j++){
			//Ordernar la línea para facilitar el "find"
			if(lineas[j].x > lineas[j].y){
				int copia_x = lineas[j].x;
				lineas[j].x = lineas[j].y;
				lineas[j].y = copia_x
			}

			if(Lineas_unicas.find(lineas[j]) != Lineas_unicas.end())
			//Si no está insertar
				Lineas_unicas.insert(lineas[j]);
			else
				//Si ya estaba borrar
				Lineas_unicas.erase(lineas[j]);
			}
	}

	if(Lineas_unicas.empty())
		return true;	//Si está vacío, el objecto está cerrado
	else
		return false;
}
```